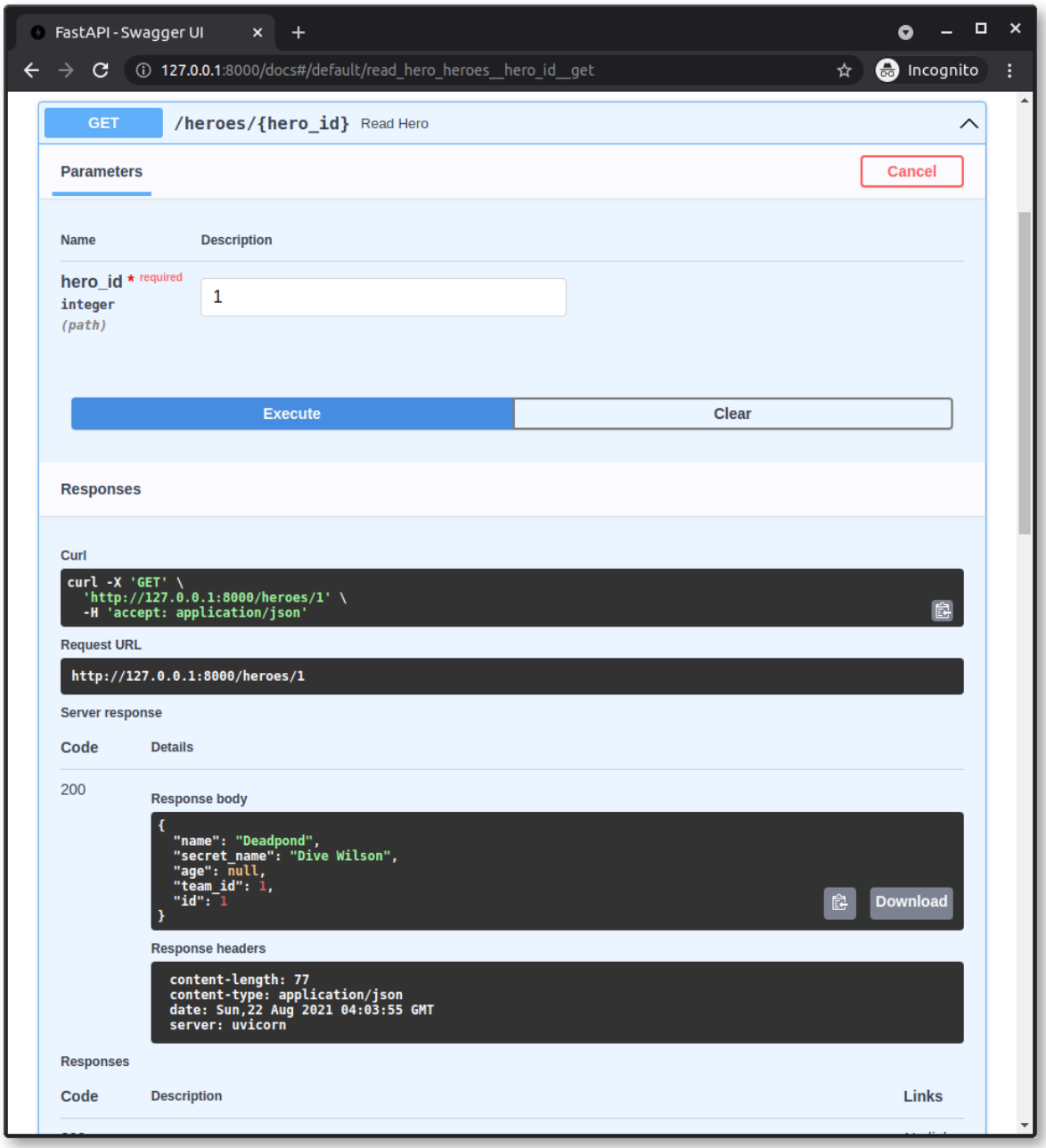
Models with Relationships in FastAPI

If we go right now and read a single **hero** by ID, we get the hero data with the team ID.

But we don't get any data about the particular team:

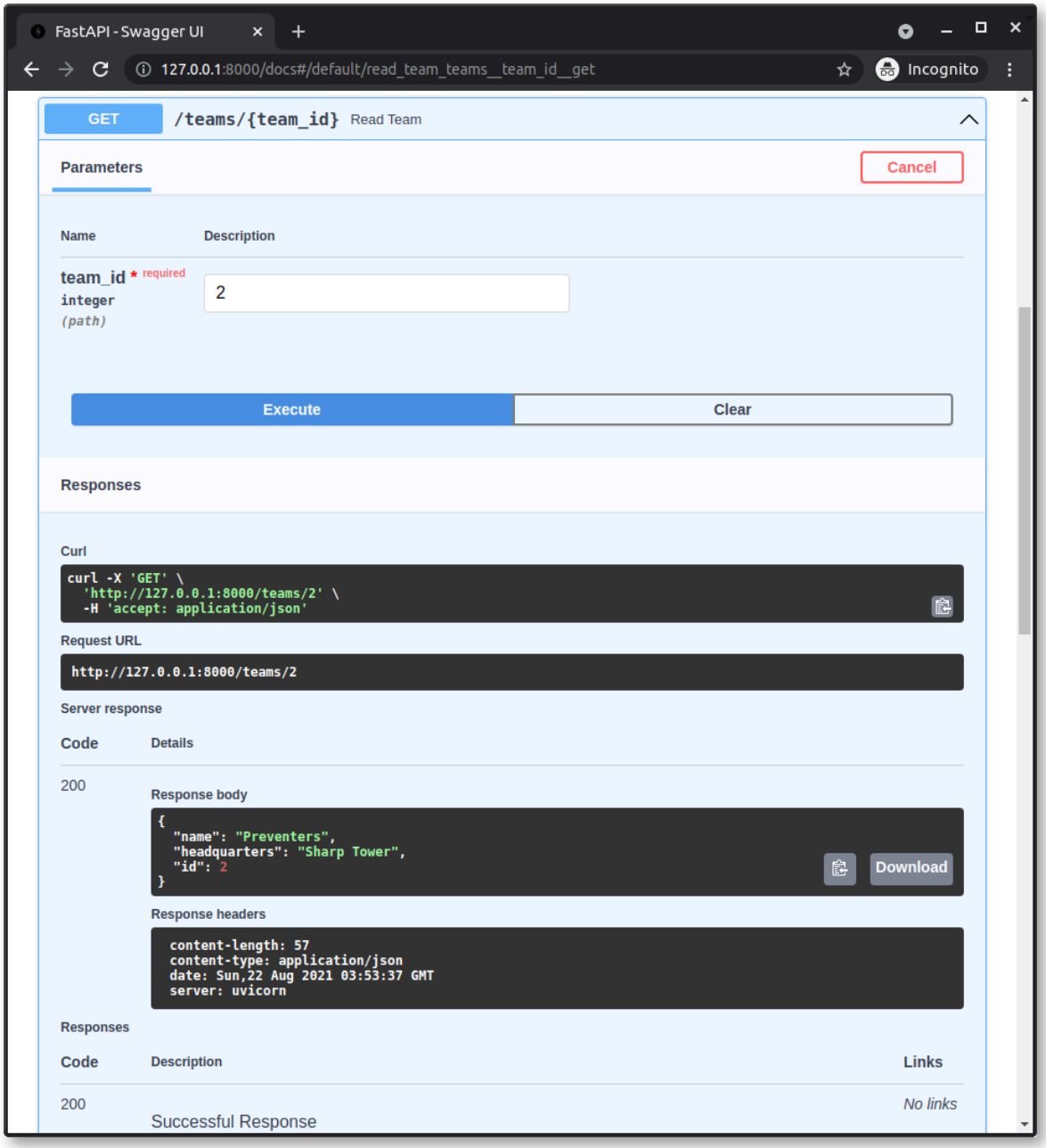


We get a response of:

```
{
    "name": "Deadpond",
    "secret_name": "Dive Wilson",
    "age": null,
```

```
    "team_id": 1,
    "id": 1,
}
```

And the same way, if we get a **team** by ID, we get the team data, but we don't get any information about this team's heroes:



Here we get a response of:

```
{
    "name": "Preventers",
    "headquarters": "Sharp Tower",
    "id": 2
}
```

...but no information about the heroes.

Let's update that. 🤓

Why Aren't We Getting More Data

First, why is it that we are not getting the related data for each hero and for each team?

It's because we declared the `HeroPublic` with only the same base fields of the `HeroBase` plus the `id`. But it doesn't include a field `team` for the **relationship attribute**.

And the same way, we declared the `TeamPublic` with only the same base fields of the `TeamBase` plus the `id`. But it doesn't include a field `heroes` for the **relationship attribute**.

Python 3.10+

```python
# Code above omitted 👆

class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str

# Code here omitted 👈

class TeamPublic(TeamBase):
    id: int

# Code here omitted 👈

class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")

# Code here omitted 👈

class HeroPublic(HeroBase):
    id: int

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
```

```python
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
```

```python
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

Now, remember that FastAPI uses the `response_model` to validate and **filter** the response data?

In this case, we used `response_model=TeamPublic` and `response_model=HeroPublic`, so FastAPI will use them to filter the response data, even if we return a **table model** that includes **relationship attributes**:

Python 3.10+

```python
# Code above omitted 👆

@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

# Code here omitted 👈

@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
```

```python
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
```

```python
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublic)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

Don't Include All the Data

Now let's stop for a second and think about it.

We cannot simply include *all* the data, including all the internal relationships, because each **hero** has an attribute `team` with their team, and then that **team** also has an attribute `heroes` with all the **heroes** in the team, including this one.

If we tried to include everything, we could make the server application **crash** trying to extract **infinite data**, going through the same hero and team over and over again internally, something like this:

```json
{
    "name": "Rusty-Man",
    "secret_name": "Tommy Sharp",
    "age": 48,
    "team_id": 1,
    "id": 1,
    "team": {
        "name": "Preventers",
        "headquarters": "Sharp Tower",
        "id": 2,
        "heroes": [
            {
                "name": "Rusty-Man",
                "secret_name": "Tommy Sharp",
                "age": 48,
                "team_id": 1,
```

```
                "id": 1,
                "team": {
                    "name": "Preventers",
                    "headquarters": "Sharp Tower",
                    "id": 2,
                    "heroes": [
                        {
                            "name": "Rusty-Man",
                            "secret_name": "Tommy Sharp",
                            "age": 48,
                            "team_id": 1,
                            "id": 1,
                            "team": {
                                "name": "Preventers",
                                "headquarters": "Sharp Tower",
                                "id": 2,
                                "heroes": [
                                    ...with infinite data here... 😱
                                ]
                            }
                        }
                    ]
                }
            }
        ]
    }
}
```

As you can see, in this example, we would get the hero **Rusty-Man**, and from this hero we would get the team **Preventers**, and then from this team we would get its heroes, of course, including **Rusty-Man**... 😱

So we start again, and in the end, the server would just crash trying to get all the data with a `"Maximum recursion error"`, we would not even get a response like the one above.

So, we need to carefully choose in which cases we want to include data and in which not.

What Data to Include

This is a decision that will depend on **each application**.

In our case, let's say that if we get a **list of heroes**, we don't want to also include each of their teams in each one.

And if we get a **list of teams**, we don't want to get a list of the heroes for each one.

But if we get a **single hero**, we want to include the team data (without the team's heroes).

And if we get a **single team**, we want to include the list of heroes (without each hero's team).

Let's add a couple more **data models** that declare that data so we can use them in those two specific *path operations*.

Models with Relationships

Let's add the models `HeroPublicWithTeam` and `TeamPublicWithHeroes`.

We'll add them **after** the other models so that we can easily reference the previous models.

Python 3.10+

```python
# Code above omitted 👆

class HeroPublicWithTeam(HeroPublic):
    team: TeamPublic | None = None
```

```python
class TeamPublicWithHeroes(TeamPublic):
    heroes: list[HeroPublic] = []

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    id: int | None = None
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


class HeroPublicWithTeam(HeroPublic):
    team: TeamPublic | None = None


class TeamPublicWithHeroes(TeamPublic):
    heroes: list[HeroPublic] = []


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
```

```python
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublicWithTeam)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
```

```
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublicWithHeroes)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

These two models are very **simple in code**, but there's a lot happening here. Let's check it out.

Inheritance and Type Annotations

The `HeroPublicWithTeam` **inherits** from `HeroPublic`, which means that it will have the **normal fields for reading**, including the required `id` that was declared in `HeroPublic`.

And then it adds the **new field** `team`, which could be `None`, and is declared with the type `TeamPublic` with the base fields for reading a team.

Then we do the same for the `TeamPublicWithHeroes`, it **inherits** from `TeamPublic`, and declares the **new field** `heroes`, which is a list of `HeroPublic`.

Data Models Without Relationship Attributes

Now, notice that these new fields `team` and `heroes` are not declared with `Relationship()`, because `HeroPublicWithTeam` and `TeamPublicWithHeroes` are not **table models**, they cannot have **relationship attributes** with the magic access to get that data from the database.

Instead, here these are only **data models** that will tell FastAPI **which attributes** to get data from and **which data** to get from them.

Reference to Other Models

Also, notice that in the `HeroPublicWithTeam` model, the field `team` is not declared with this new `TeamPublicWithHeroes`, because that would again create that infinite recursion of data. Instead, we declare it with the normal `TeamPublic` model.

And the same for `TeamPublicWithHeroes`, the model used for the new field `heroes` uses `HeroPublic` to get only each hero's data.

This also means that, even though we have these two new models, **we still need the previous ones**, `HeroPublic` and `TeamPublic`, because we need to reference them here (and we are also using them in the rest of the *path operations*).

Update the Path Operations

Now we can update the *path operations* to use the new models.

This will tell **FastAPI** to take the object that we return from the *path operation function* (a **table model**) and **access the additional attributes** from them to extract their data.

In the case of the hero, this tells FastAPI to extract the `team` too. And in the case of the team, to extract the list of `heroes` too.

Python 3.10+

```python
# Code above omitted 👆

@app.get("/heroes/{hero_id}", response_model=HeroPublicWithTeam)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

# Code here omitted 👈

@app.get("/teams/{team_id}", response_model=TeamPublicWithHeroes)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class TeamBase(SQLModel):
    name: str = Field(index=True)
    headquarters: str


class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    heroes: list["Hero"] = Relationship(back_populates="team")


class TeamCreate(TeamBase):
    pass


class TeamPublic(TeamBase):
    id: int


class TeamUpdate(SQLModel):
    id: int | None = None
    name: str | None = None
    headquarters: str | None = None


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

    team: Team | None = Relationship(back_populates="heroes")


class HeroPublic(HeroBase):
    id: int


class HeroCreate(HeroBase):
    pass


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    team_id: int | None = None


class HeroPublicWithTeam(HeroPublic):
    team: TeamPublic | None = None


class TeamPublicWithHeroes(TeamPublic):
    heroes: list[HeroPublic] = []


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
```

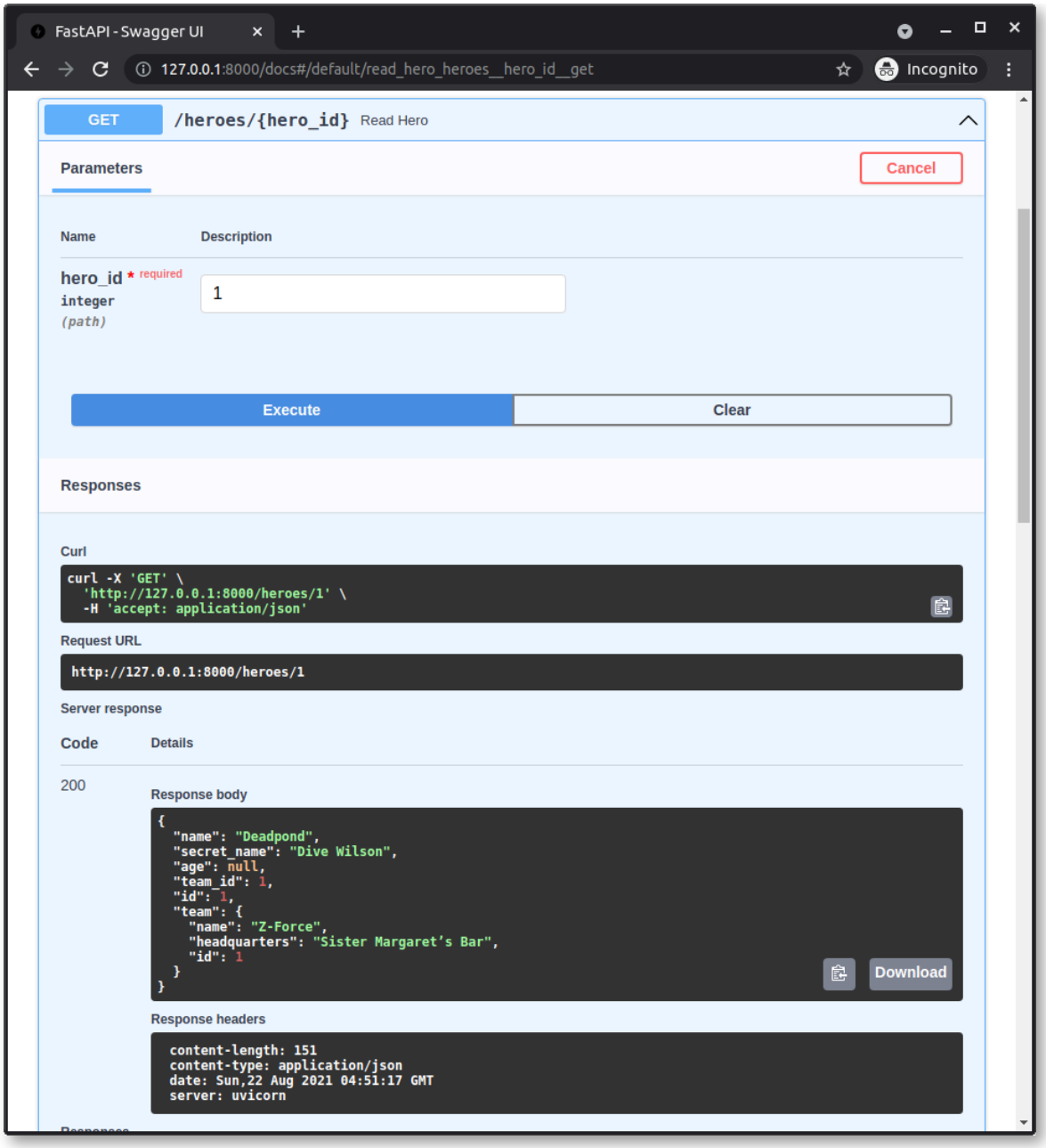
```python
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def get_session():
    with Session(engine) as session:
        yield session


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(*, session: Session = Depends(get_session), hero: HeroCreate):
    db_hero = Hero.model_validate(hero)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
    return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublicWithTeam)
def read_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(
    *, session: Session = Depends(get_session), hero_id: int, hero: HeroUpdate
):
    db_hero = session.get(Hero, hero_id)
    if not db_hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    hero_data = hero.model_dump(exclude_unset=True)
    db_hero.sqlmodel_update(hero_data)
    session.add(db_hero)
    session.commit()
    session.refresh(db_hero)
    return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(*, session: Session = Depends(get_session), hero_id: int):
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    session.delete(hero)
    session.commit()
    return {"ok": True}


@app.post("/teams/", response_model=TeamPublic)
def create_team(*, session: Session = Depends(get_session), team: TeamCreate):
    db_team = Team.model_validate(team)
```

```python
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.get("/teams/", response_model=list[TeamPublic])
def read_teams(
    *,
    session: Session = Depends(get_session),
    offset: int = 0,
    limit: int = Query(default=100, le=100),
):
    teams = session.exec(select(Team).offset(offset).limit(limit)).all()
    return teams


@app.get("/teams/{team_id}", response_model=TeamPublicWithHeroes)
def read_team(*, team_id: int, session: Session = Depends(get_session)):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team


@app.patch("/teams/{team_id}", response_model=TeamPublic)
def update_team(
    *,
    session: Session = Depends(get_session),
    team_id: int,
    team: TeamUpdate,
):
    db_team = session.get(Team, team_id)
    if not db_team:
        raise HTTPException(status_code=404, detail="Team not found")
    team_data = team.model_dump(exclude_unset=True)
    db_team.sqlmodel_update(team_data)
    session.add(db_team)
    session.commit()
    session.refresh(db_team)
    return db_team


@app.delete("/teams/{team_id}")
def delete_team(*, session: Session = Depends(get_session), team_id: int):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    session.delete(team)
    session.commit()
    return {"ok": True}
```

Check It Out in the Docs UI

Now let's try it out again in the **docs UI**.

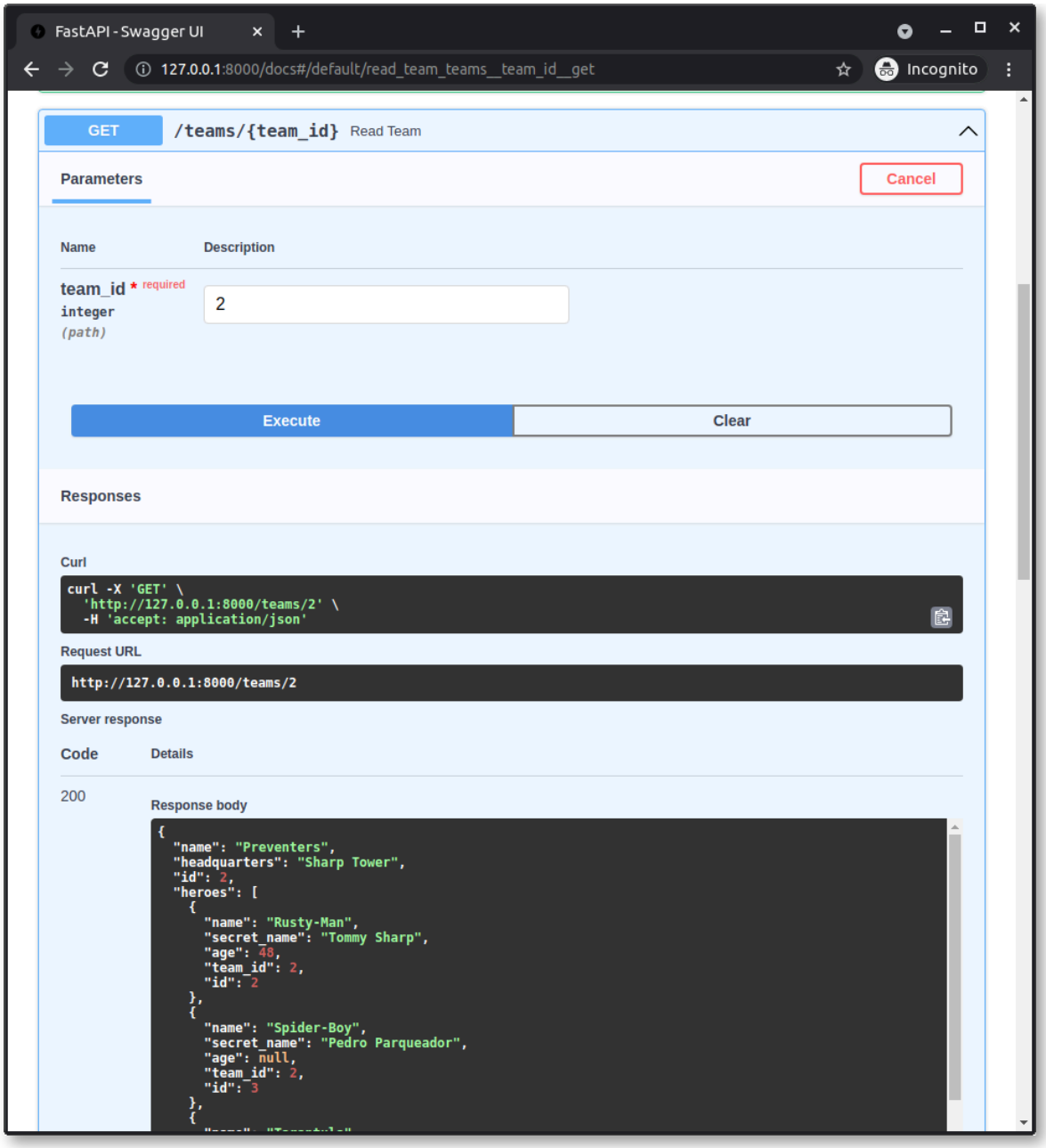
Let's try again with the same **hero** with ID `1`:



Now we get the **team** data included:

```
{
    "name": "Deadpond",
    "secret_name": "Dive Wilson",
    "age": null,
    "team_id": 1,
    "id": 1,
    "team": {
        "name": "Z-Force",
        "headquarters": "Sister Margaret's Bar",
        "id": 1
    }
}
```

And if we get now the **team** with ID `2`:



Now we get the list of **heroes** included:


```
{
    "name": "Preventers",
    "headquarters": "Sharp Tower",
    "id": 2,
    "heroes": [
        {
            "name": "Rusty-Man",
            "secret_name": "Tommy Sharp",
            "age": 48,
            "team_id": 2,
            "id": 2
        },
        {
            "name": "Spider-Boy",
            "secret_name": "Pedro Parqueador",
            "age": null,
            "team_id": 2,
            "id": 3
        },
```

```
        {
            "name": "Tarantula",
            "secret_name": "Natalia Roman-on",
            "age": 32,
            "team_id": 2,
            "id": 6
        },
        {
            "name": "Dr. Weird",
            "secret_name": "Steve Weird",
            "age": 36,
            "team_id": 2,
            "id": 7
        },
        {
            "name": "Captain North America",
            "secret_name": "Esteban Rogelios",
            "age": 93,
            "team_id": 2,
            "id": 8
        }
    ]
}
```

Recap

Using the same techniques to declare additional **data models**, we can tell FastAPI what data to return in the responses, even when we return **table models**.

Here we almost **didn't have to change the FastAPI app** code, but of course, there will be cases where you need to get the data and process it in different ways in the *path operation function* before returning it.

But even in those cases, you will be able to define the **data models** to use in `response_model` to tell FastAPI how to validate and filter the data.

By this point, you already have a very robust API to handle data in a SQL database combining **SQLModel** with **FastAPI**, and implementing **best practices**, like data validation, conversion, filtering, and documentation. ✨

In the next chapter, I'll tell you how to implement automated **testing** for your application using FastAPI and SQLModel. ✅